

53. Une fonction qui rend une valeur

Ce qui entre, ce qui sort — et ce qu'on ne réécrit pas deux fois.

1. Ce qu'il faut comprendre

Les deux appels rendent 70. Sans la fonction, il faudrait écrire le `if` deux fois — et le jour où l'on veut borner l'écart à 60, le corriger aux deux endroits.

Le résultat s'utilise dans un calcul : `nombre(10, 6, distance(a, b) * 2)` marche, comme n'importe quelle expression.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1365 octets** de cartouche.

1. UNE FONCTION — DISTANCE()

```
uint8_t distance(uint8_t a, uint8_t b) {  
    if (a > b) return a - b;  
    return b - a;  
}
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

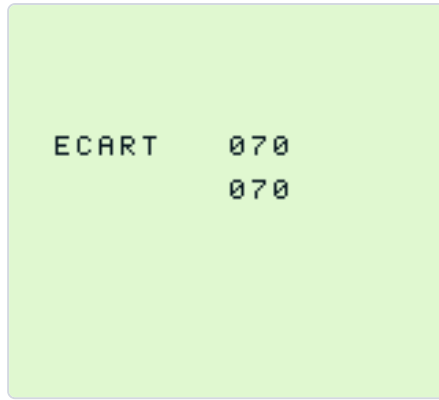
```
int main() {  
    texte(2, 6, "ECART");  
    nombre(10, 6, distance(30, 100));  
    nombre(10, 8, distance(100, 30));  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

070 deux fois — l'ordre des arguments ne change rien.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le premier sens donne 70
- ✓ le second aussi
- ✓ l'étiquette est là

Ensuite : **54. Un argument par défaut** — niveau 7 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++